# UMINE

## Plan de Acción Post-Reunión

09 de Febrero 2026

| | |
|---|---|
| **Participantes:** | Leonardo Cortes, Raimundo Burchardt |
| **Contexto:** | Revisión de avances y asignación de tareas |
| **Documento:** | Accionable dividido en 2 etapas |

**Etapa 1: Tareas asignadas por Leonardo Cortes**
**Etapa 2: Core de PMO / Gestión de Productos (Raimundo)**

# Resumen Ejecutivo

Este documento consolida los acuerdos y tareas definidas en la reunión del 9 de febrero de 2026 entre Leonardo Cortes (Tech Advisor) y Raimundo Burchardt. Se divide en dos etapas claras de ejecución.

**Etapa 1** agrupa las tareas que Leonardo Cortes asignó directamente a Raimundo: clonar el repositorio de Operaciones, migrar Firebase a AWS (DynamoDB + Lambda), y adaptar el layout del Core de PM al estándar de la plataforma.

**Etapa 2** cubre la construcción del Core de PMO propio de Raimundo: la plataforma de gobernanza con gestión de solicitudes, user stories, Kanban, workspaces y changelog, integrada al ecosistema Umine.

# Contexto de Arquitectura Umine

La plataforma Umine se organiza en múltiples “Cores” desplegados en AWS con arquitectura serverless. Leonardo Cortes lidera el desarrollo de la infraestructura base y los siguientes módulos ya están en construcción o activos:

| Core | Descripción | Estado |
|---|---|---|
| **Core Comercial** | Gestión de ventas y SDR | En desarrollo |
| **Core Contenido** | Configurador de encuestas clima, cursos, branding, generación IA | Frontend avanzado, falta backend |
| **Core DTI** | Soporte ITIL, tickets, gestión de proyectos | Frontend listo, falta backend |
| **Core Operaciones** | Mantenedor clientes, OC, integraciones SENCE, dashboard | En construcción activa (proyecto Claudia) |
| **Smart Brand Builder** | Configurador de marca con export JSON para branding de apps | Operativo |
| **Core Autorización** | Login Google + capa de acceso centralizada por plataforma/rol | Leo lo construye esta semana |

Stack tecnológico definido: AWS Lambda (serverless), DynamoDB (tablas globales replicadas en 2 zonas US East/West), API Gateway, Amplify para frontends, DNS para balanceo de carga. Todo el backend se centraliza en AWS para estandarizar costos y evitar fragmentación (Firebase queda descartado como DB).

# ETAPA 1: Tareas Asignadas por Leonardo Cortes

Estas son las tareas que Leonardo Cortes definió explícitamente para Raimundo durante la reunión. Cada tarea tiene dependencias claras y un entregable concreto.

## 1.1 Clonar Repositorio de Operaciones

| | |
|---|---|
| **Responsable** | Raimundo Burchardt |
| **Prioridad** | **ALTA — Bloqueante para Etapa 2** |
| **Prerequisito** | Leo da acceso al repo de Operaciones en GitHub |
| **Estado** | Invitación enviada a usuario Git de Raimundo |

**Pasos a ejecutar:**

- Hacer git clone del repositorio de Operaciones.
- Eliminar la carpeta .git del proyecto clonado para desvincularlo del repo original.
- Crear un nuevo repositorio con el estándar de naming: ui-core-prog-managers (o ui-core-pm).
- Subir el proyecto a AWS Amplify.
- Notificar a Leo para que agregue la ruta al Changelog general de la plataforma.

**Entregable:** Repositorio ui-core-prog-managers inicializado, con el layout de Operaciones como base, desplegado en Amplify y registrado en el Changelog.

## 1.2 Migrar Backend de Firebase a AWS

| | |
|---|---|
| **Responsable** | Raimundo Burchardt |
| **Prioridad** | **ALTA — Alineación arquitectural obligatoria** |
| **Razón** | Toda la arquitectura se estandariza en AWS para costos, escalabilidad e integración entre Cores |

**Lo que Leo indicó explícitamente:**

*"Saca las tablas de Firebase, colócalas en DynamoDB y expone los métodos a través de Lambda. En vez de consumir la librería de Firebase desde el frontend, vas a consumir la API de AWS. Elimina Firebase como base de datos, todo en DynamoDB para que lo podamos apilar."*

**Pasos a ejecutar:**

- Exportar las tablas/colecciones actuales de Firebase (estructura + datos).
- Diseñar las tablas equivalentes en DynamoDB (partition key, sort key, GSIs).

- Crear funciones Lambda para exponer los métodos CRUD vía API Gateway.
- Reemplazar en el frontend todas las llamadas a Firebase SDK por llamadas a la API de AWS.
- Eliminar dependencia de Firebase del proyecto (desinstalar SDK).
- Validar que el login con Google siga funcionando (el auth puede mantenerse separado inicialmente).

**Tip de Leo:** "Le puedes decir a Claude: llévame Firebase a Lambda y genérame las Lambda y dame la estructura de tabla, y te lo hace al toque."

**Entregable:** Backend 100% en AWS (DynamoDB + Lambda + API Gateway). Cero dependencia de Firebase como DB. Frontend consumiendo API de AWS.

## 1.3 Adaptar Layout al Estándar Visual de la Plataforma

| | |
|---|---|
| **Responsable** | Raimundo Burchardt |
| **Prioridad** | **MEDIA — Consistencia visual entre Cores** |
| **Método** | Copiar proyecto completo de la app actual a una subcarpeta del proyecto clonado y fusionar |

### Lo que Leo indicó:

*"Cópiate tu proyecto completo de la app y colócalo dentro del proyecto que clonaste. Decíle a Claude: quiero implementar toda la funcionalidad dentro de este nuevo proyecto, pero sin afectar el login ni la imagen visual, que se creen como funcionalidades."*

### Criterios de aceptación:

- El Core de PM debe verse visualmente idéntico al resto de Cores (mismo sidebar, header, estilos).
- Login con Google debe seguir funcionando sin afectarse.
- Las funcionalidades de gobernanza (Kanban, workspaces, user stories) deben integrarse como secciones dentro del layout.
- Incluir un Changelog en formato JSON para que el servicio centralizado pueda leerlo y exponer novedades.

## 1.4 Consumir API Interna de Operaciones

| | |
|---|---|
| **Responsable** | Raimundo Burchardt (con coordinación de Leo) |
| **Prioridad** | **MEDIA — Depende de que Leo termine la API de Operaciones** |
| **Contexto** | El Core de Operaciones tiene API interna + API externa. Otros Cores consumen la API externa para datos de clientes. |

### Endpoints conocidos de la API de Operaciones:

- GET /customers — Obtener listado de clientes
- POST /customers — Crear cliente
- PUT /customers/:id — Actualizar cliente
- Métodos adicionales por definir en coordinación con Leo.

Cuando Leo tenga la API lista, configurar el API Context en el frontend del Core PM para apuntar a estos endpoints. Inicialmente trabajar en local sin integraciones, con datos mock.

## 1.5 Tareas que Leo Ejecuta en Paralelo (Referencia)

Estas tareas las ejecuta Leonardo Cortes. Se listan como referencia para entender dependencias y bloqueos.

| Tarea de Leo | Impacto en Raimundo | Timeline Estimado |
|---|---|---|
| Construir Core de Autorización | Habilita control de acceso por plataforma y rol para el Core PM | Esta semana (tabla global DynamoDB replicada US East/West) |
| Completar proyecto Claudia (Core Ops) | Define la API externa que Raimundo consumirá desde su Core PM | "En un día lo saco" (backend de OC + dashboard) |
| Balanceo de carga serverless | Mejora disponibilidad de toda la plataforma, incluyendo Core PM | Post-autorización (DynamoDB Global Tables + DNS) |
| Servicio de email vía API | Permite notificaciones desde Core PM (ej: nueva solicitud asignada) | En evaluación (envío en tiempo real vs cola diaria) |
| Subir portal cliente con Encuesta Clima | No bloquea a Raimundo directamente | En paralelo |

# ETAPA 2: Core de PMO — Plataforma de Gestión de Productos

Esta etapa cubre la evolución de la herramienta de gobernanza propia de Raimundo. El objetivo es convertirla en el Core de Producto (Prog Managers) integrado al ecosistema Umine, resolviendo el problema de que las solicitudes actualmente se manejan por email y chat sin trazabilidad.

## 2.1 Problema que Resuelve

Actualmente las solicitudes de producto en Umine se manejan de forma dispersa: llegan por email, chat, y no hay un backlog centralizado. Esto provoca pérdida de contexto, duplicación de esfuerzo y falta de visibilidad del estado de cada requerimiento.

**Capacidades actuales de la plataforma de Raimundo:**

- Dashboard de gobernanza.
- Tablero Kanban para tareas.
- Workspace para crear solicitudes con artefactos (HTML desde Claude).
- Creación de user stories con backlog.
- Single Sign-On con Google.
- Actualmente desplegado en Firebase (producción).

## 2.2 Módulos del Core PMO

| Módulo | Funcionalidad | Estado Actual | Prioridad |
|---|---|---|---|
| **Dashboard** | Vista general de proyectos, métricas y estado global del backlog | Existe (por adaptar al layout) | MEDIA |
| **Kanban** | Tablero visual de tareas con drag & drop, asignación de responsables, etiquetas | Existe (funcional) | ALTA |
| **Workspaces** | Espacio por proyecto/área donde se centralizan solicitudes. Ej: “Mejoras Clima”. Permite adjuntar artefactos de Claude (HTML copiado) | Existe (funcional) | ALTA |
| **User Stories** | Creación de historias: “Como [rol] necesito [acción] para [beneficio]” con criterios de aceptación | Existe (funcional) | ALTA |
| **Changelog** | JSON con novedades del Core para que el servicio centralizado lo lea y notifique cambios | Nuevo (por crear) | ALTA |
| **Integraciones** | Consumir API externa de Operaciones (clientes) + futuro Core de Autorización | Nuevo (por crear) | MEDIA |

## 2.3 Plan de Implementación del Core PMO

### Fase A: Migración y Layout (Semana 1-2)

| # | Tarea | Tipo | Status |
|---|---|---|---|
| A1 | Git clone del repo de Operaciones + eliminar .git | Setup | PENDIENTE |
| A2 | Crear repo ui-core-prog-managers con naming estándar | Setup | PENDIENTE |
| A3 | Copiar código de la app de gobernanza actual a subcarpeta del nuevo proyecto | Migración | PENDIENTE |
| A4 | Fusionar funcionalidades al layout de Operaciones (sidebar, header, estilos) | Frontend | PENDIENTE |
| A5 | Validar login Google no se afecta tras la fusión | QA | PENDIENTE |
| A6 | Deploy a Amplify + notificar a Leo para Changelog | Deploy | PENDIENTE |

### Fase B: Migración de Backend Firebase a AWS (Semana 2-3)

| # | Tarea | Tipo | Status |
|---|---|---|---|
| B1 | Exportar esquema y datos de Firebase (Firestore/Realtime DB) | Data | PENDIENTE |
| B2 | Diseñar tablas DynamoDB (PK, SK, GSIs) para cada colección | Arquitectura | PENDIENTE |
| B3 | Crear Lambdas CRUD + API Gateway para cada entidad | Backend | PENDIENTE |
| B4 | Reemplazar Firebase SDK por llamadas a API AWS en frontend | Frontend | PENDIENTE |
| B5 | Migrar datos de producción de Firebase a DynamoDB | Data | PENDIENTE |
| B6 | Eliminar dependencia de Firebase del proyecto | Cleanup | PENDIENTE |
| B7 | Testing end-to-end en AWS | QA | PENDIENTE |

### Fase C: Integración con Ecosistema Umine (Semana 3-4)

| # | Tarea | Tipo | Status |
|---|---|---|---|
| C1 | Crear changelog.json con estructura estándar para novedades | Feature | PENDIENTE |
| C2 | Integrar API externa de Operaciones (datos de clientes) cuando esté disponible | Integración | BLOQUEADO |
| C3 | Integrar Core de Autorización (control de acceso por rol/plataforma) | Integración | BLOQUEADO |
| C4 | Configurar API Context en frontend apuntando a | Frontend | PENDIENTE |

| | | | |
|---|---|---|---|
| | métodos reales | | |
| C5 | Integrar notificaciones vía servicio de email de Leo (cuando esté listo) | Integración | **BLOQUEADO** |

## 2.4 Flujo de Uso Esperado del Core PMO

**El flujo end-to-end de cómo un área de Umine (por ejemplo, Begoña pidiendo mejoras en Encuesta de Clima) usaría la plataforma:**

| Paso | Actor | Acción |
|---|---|---|
| 1 | **Solicitante** | Ingresa al Core PMO, selecciona Workspace (ej: “Mejoras Clima”) |
| 2 | **Solicitante** | Crea una solicitud con descripción del requerimiento. Opcionalmente pega un artefacto HTML generado con Claude como especificación visual. |
| 3 | **Product Manager** | Revisa la solicitud, la refina y la convierte en una o más User Stories con criterios de aceptación. |
| 4 | **Product Manager** | Agrega las User Stories al Backlog y las prioriza en el Kanban (To Do, In Progress, Done). |
| 5 | **Desarrollador** | Toma tareas del Kanban, las ejecuta y actualiza el estado. |
| 6 | **Sistema** | Al cerrar una historia, se registra en el Changelog automáticamente para que otros Cores puedan mostrar las novedades. |

# Timeline Consolidado

| Semana | Foco Principal | Etapa | Dependencia | Riesgo |
|---|---|---|---|---|
| **S1** | Clone repo + Setup proyecto + Eliminar .git + Nuevo repo | **ETAPA 1** | Acceso GitHub | **BAJO** |
| **S1-S2** | Fusionar funcionalidades al layout + Validar login + Deploy Amplify | **ETAPA 1+2** | Ninguna | **BAJO** |
| **S2-S3** | Exportar Firebase + Diseñar DynamoDB + Crear Lambdas + Migrar | **ETAPA 1** | Ninguna | **MEDIO** |
| **S3-S4** | Changelog + Integración APIs (Ops, Auth, Email) | **ETAPA 2** | Leo: API Ops + Auth | **MEDIO** |

# Decisiones Clave de la Reunión

| # | Decisión | Justificación |
|---|---|---|
| 1 | **Firebase queda descartado como base de datos. Todo a DynamoDB en AWS.** | Estandarización de arquitectura, reducción de costos, evitar integraciones cruzadas entre proveedores. |
| 2 | **El Core de PM de Raimundo se integra al ecosistema Umine con el layout estándar.** | Consistencia visual, experiencia unificada para usuarios internos, reutilización de componentes. |
| 3 | **Login Google se mantiene abierto (no restringido a dominio @umine). La autorización se maneja en capa separada.** | Permite acceso a clientes externos en el futuro. La capa de autorización valida post-login qué plataformas puede ver cada usuario. |
| 4 | **Cada Core debe tener un Changelog en JSON.** | Permite al servicio centralizado leer novedades de todos los Cores y generar notificaciones automáticas. |
| 5 | **DynamoDB Global Tables (replicación US East + US West) para balanceo de carga serverless.** | Alta disponibilidad, failover automático vía DNS. Costo justificado por estar en free tier con pocos clientes. |

# Checklist Rápido para Raimundo

| ☐ | Tarea | Bloqueo |
|---|---|---|
| ☐ | Aceptar invitación al repo de Operaciones en GitHub | **LIBRE** |
| ☐ | Git clone + eliminar .git + crear ui-core-prog-managers | **LIBRE** |
| ☐ | Copiar app de gobernanza a subcarpeta del proyecto | **LIBRE** |
| ☐ | Fusionar al layout estándar sin afectar login ni imagen visual | **LIBRE** |
| ☐ | Deploy a Amplify + avisar a Leo | **LIBRE** |
| ☐ | Exportar tablas de Firebase | **LIBRE** |
| ☐ | Diseñar DynamoDB + crear Lambdas + migrar datos | **LIBRE** |

| | | |
|---|---|---|
| ☐ | Reemplazar Firebase SDK por API AWS en frontend | **LIBRE** |
| ☐ | Crear changelog.json con estructura estándar | **LIBRE** |
| ☐ | Integrar API de Operaciones (clientes) | **ESPERA LEO** |
| ☐ | Integrar Core de Autorización | **ESPERA LEO** |
| ☐ | Integrar servicio de email para notificaciones | **ESPERA LEO** |